Chapter 16 -- Conceptual Modeling: Building the First Semantic Taxonomy

Semantic Knowledge Development Lifecycle (SKDL) - Stage 1: Conceptual Modeling

Every semantic system begins with a well-structured conceptual model.

16.1 Introduction -- Conceptual Modeling: The Foundation of Semantic Knowledge

Every engineering and architectural discipline begins with **abstraction**.

- Before software engineers implement classes and algorithms, they first identify the fundamental concepts that make up the problem domain.
- Before database designers normalize tables and define constraints, they establish entities and relationships.
- Before enterprise architects analyze systems and processes, they develop conceptual models describing business capabilities, information assets, and organizational structures.

Ontology engineering follows exactly the same engineering philosophy.

Before semantic relationships can be established, before logical restrictions can be defined, before automated reasoning can infer new knowledge, and before intelligent systems can consume semantic information, an ontology must first establish a coherent conceptual representation of the domain it intends to describe.

This activity is known as **Conceptual Modeling**, the first stage of **Semantic Knowledge Development Lifecycle (SKDL)** introduced in Chapter (15).

Conceptual Modeling addresses one of the most fundamental questions in knowledge engineering:

> **What concepts exist within the domain, and how should they be organied into a meaningful semantic hierarchy?**

Although this question appears deceptively simple, its answer determines the quality of every subsequent stage of ontology development.

A poorly designed conceptual model inevitably produces ambiguous semantics, inconsistent reasoning, and difficult-to-maintain ontologies. Conversely, a well-designed conceptual hierarchy provides a stable semantic foundation upon which logical definitions, reasoning rules, governance policies, and executable knowledge can progressively be constructed.

This engineering philosophy explains the structure of Michael DeBellis' `Pizza` tutorial.

Having introduced the core language of OWL throughout the previous exercises, the tutorial now transitions from learning individual ontology constructs to building a complete semantic model.

Rather than immediately introducing more sophisticated logical expressions, **Exercise 14** begins by expanding the conceptual hierarchy of the `Pizza` ontology through the creation of its first meaningful subclasses.

This design reflects an important engineering principle that extends far beyond the `Pizza` example:

> **Semantic meaning should be constructed upon conceptual organization -- not the other way around!**

Throughout this chapter, we will examine how conceptual modeling establishes the semantic vocabulary of an ontology, why taxonomies are essential for organizing domain knowledge, and how this first stage of the Semantic Knowledge Development Lifecycle lays the foundation for every subsequent modeling activity.

16.2 Learning Objectives

After completing this chapter, you should be able to:

- Explain the purpose of **Conceptual Modeling** within the Semantic Knowledge Development Lifecycle.
- Understand why ontology engineering begins by identifying concepts before defining semantic behavior.
- Create and organize semantic concepts using subclass hierarchies in Protégé.
- Distinguish between abstraction and specialization within an ontology.
- Explain the role of semantic taxonomies in supporting scalable ontology design.
- Recognize how conceptual modeling contributes to the K **- Knowledge Graph** layer of the EKA framework.
- Appreciate why conceptual modeling influences reasoning, governance, validation, and future ontology evolution.

16.3 Exercise 14 -- Establishing the First Semantic Taxonomy

From the perspective of a Protégé user, Exercise 14 appears relatively straightforward. Two new classes -- `NamedPizza` and `MargheritaPizza` -- are created beneath the existing `Pizza` class.

From the perspective of ontology engineering, however, this exercise represents the first deliberate act of **semantic abstraction.**

Until this point, the `Pizza` ontology primarily consists of general concepts describing pizzas, toppings, ingredients, and bases. While these concepts establish the vocabulary of the domain, they have not yet been organized into a reusable conceptual hierarchy capable of supporting *semantic inheritance and future reasoning*.

Exercise 14 begins transforming this collection of concepts into a structured taxonomy.

The resulting hierarchy is illustrated below:

```
/
└ Thing
  └ Food
    └ Pizza
      └ NamedPizza
        └ MargheritaPizza
```

Although only two additional classes are introduced, they fundamentally change how knowledge is organized within the ontology.

Rather than treating every pizza as a direct specialization of `Pizza`, the ontology now introduces multiple levels of semantic abstraction.

This design provides a clear separation between general concepts and increasingly specialized domain concepts.

`Pizza` -- The General Concept

The class `Pizza` represents the generic semantic concepts of a pizza.

At this level, the ontology deliberately avoids describing specific recipes, commercial products, or individual menu items.

Instead, the class captures only those characteristics that are common to every pizza.

By maintaining this high level of abstraction, future subclasses can inherit these common semantics without unnecessary duplication.

This approach follows one of the central principles of ontology engineering:

> **Common knowledge should be modeled once and inherited wherever possible.**

`NamedPizza` -- Introducing an Intermediate Abstraction

Many beginners initially question why the tutorial introduces `NamedPizza` rather than placing `MargheritaPizza` directly beneath `Pizza`.

The answer lies in the importance of **abstraction**.

`NamedPizza` does not represent a particular pizza.

Instead, it represents an entire category of pizzas that posses recognized names within the domain.

Examples include:

- `MargheritaPizza`

- `AmericanaPizza`
- `SohoPizza`
- `FiorentinaPizza`
- `VenezianaPizza`

Introducing this intermediate concept provides several engineering advantages.

1. It groups all commercially recognized ("Named") pizzas into a coherent semantic category.
2. It prevents the `Pizza` hierarchy from becoming unnecessarily crowded as additional pizza varieties are introduced.
3. It creates a reusable abstraction upon which future semantic definitions can be consistently applied.

As ontologies continue to evolve, intermediate abstractions such as `NamedPizza` become increasingly valuable because they **reduce redundancy** while **improving maintainability**.

`MargheritaPizza` -- The First Concrete Domain Concept

The class `MargheritaPizza` represents the first concrete pizza type introduced within the ontology.

Unlike `NamedPizza`, which functions primarily as an organizational abstraction, `MargheritaPizza` corresponds to a specific pizza variety that people recognize in everyday life.

At this stage of the tutorial, however, the ontology intentionally refrains from describing what makes a Margherita pizza unique.

It does not yet specify:

- which toppings it contains;
- which ingredients distinguish it from other pizzas; or
- which logical restrictions formally define it.

These semantic descriptions belong to the next stage of the Semantic Knowledge Development Lifecycle.

For now, the objective is considerably simpler:

> **Identify the concept before describing its meaning.**

This separation between conceptual identification and semantic definition represents one of the defining characteristics of professional ontology engineering.

An Engineering Perspective

Viewed purely as a Protégé exercise, Exercise 14 consists of creating two subclasses.

Viewed through the lens of ontology engineering, however, it marks the beginning of systematic semantic knowledge development.

The ontology is no longer expanding by adding isolated concepts.

Instead, it is establishing a structured conceptual vocabulary capable of supporting:

- semantic inheritance;
- logical specialization;

- reusable ontology patterns;
- automated reasoning;
- semantic governance; and
- future knowledge graph development.

Although the modeling effort appears modest, the architectural significance is substantial.

Exercise 14 therefore represents the true starting point of conceptual modeling within the `Pizza` ontology.

16.4 Why Conceptual Modeling Comes Before Semantic Modeling

A common misconception among newcomers to ontology engineering is that the primary objective of an ontology is to define logical rules.

In reality, logical semantics represent only one stage of ontology development.

Before logical rules can be introduced, the ontology must first establish a stable conceptual structure capable of supporting those rules.

This design philosophy is shared across virtually every engineering discipline.

Discipline	Primary Question	Initial Modeling Activity
Software Engineering	What objects exist?	Class Modeling
Database Design	What information should be stored?	Entity-Relationship Modeling
Business Architecture	What business capabilities exist?	Capability Mapping
Enterprise Architecture	What architectural elements exist?	Architecture Modeling
Ontology Engineering	What concepts exist?	Conceptual Modeling

Although the terminology differs, the underlying engineering principle remains remarkably consistent.

Every discipline begins by identifying **what exists** before attempting to describe **how it behaves.**

Ontology engineering follows exactly the same progression.

Conceptual Modeling therefore answers only one fundamental questions:

> **What concepts exist within the knowledge domain?**

Later stages of the Semantic Knowledge Development Lifecycle answer progressively model sophisticated questions.

For example:

- How are concepts semantically related?
- Which logical constraints govern them?
- Which facts can be inferred automatically?
- How should semantic quality be validated?
- How can semantic knowledge drive enterprise execution?

Attempting to answer these questions before a stable conceptual model exists often leads to ontologies that are inconsistent, difficult to understand, and challenging to maintain.

For this reason, professional ontology engineers invest considerable effort in conceptual modeling before introducing more sophisticated semantic constructs.

16.5 Building a Semantic Taxonomy

Once the primary concepts of a domain have been identified, they must be organized into a coherent semantic hierarchy.

This hierarchy is known as a **taxonomy**.

Although taxonomies are often visualized as tree structures, their purpose extends far beyond simple organization.

A semantic taxonomy captures the **generalization-specialization relationships** between concepts.

Each child concept represents a more specialized interpretation of its parent while inheriting all of the semantic meaning already established at higher levels.

The `Pizza` ontology now contains the following conceptual hierarchy.

$$Thing \rightarrow Food \rightarrow Pizza \rightarrow NamedPizza \rightarrow MargheritaPizza$$

Each successive level introduces additional semantic specificity.

Moving downward through the hierarchy does not create unrelated concepts.

Instead, every specialization preserves the semantic identity of its ancestors.

Consequently:

- Every `MargheritaPizza` is a `NamedPizza`.
- Every `NamedPizza` is a `Pizza`.
- Every `Pizza` is `Food`. (although we don't create this layer in our working model)
- Every `Food` is ultimately a `Thing`.

This hierarchical organization provides numerous engineering benefits.

A well-designed taxonomy improves readability by presenting domain concepts in a logical and intuitive structure.

It improves maintainability by allowing common characteristics to be inherited rather than repeatedly modeled.

It enhances extensibility by enabling new concepts to be incorporated without disrupting the overall organization of the ontology.

Most importantly, the taxonomy establishes the conceptual backbone upon which subsequent stages of the Semantic Knowledge Development Lifecycle will build increasingly sophisticated semantic capabilities.

Without a robust conceptual hierarchy:

- logical restrictions become difficult to define,
- reasoning becomes less effective,
- governance becomes harder to enforce, and
- semantic reuse becomes increasingly limited.

For this reason, conceptual modeling should never be regarded as a preliminary modeling exercise.

It is the architectural foundation upon which every successful ontology -- and ultimately every executable knowledge system -- is built.

16.6 Taxonomy Is More Than a Tree

When first learning Protégé, many beginners perceive the class hierarchy as little more than a graphical tree used to organize concepts.

From this perspective, subclasses resemble folders within a file system, where each child node is simply grouped beneath a parent node for convenience.

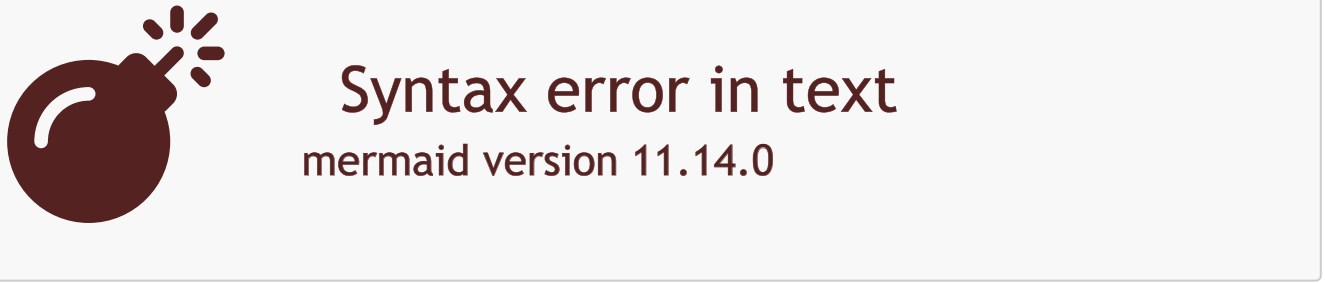
This interpretation, however, fundamentally misunderstands the purpose of an ontology.

A folder hierarchy is primarily an organizational mechanism. It helps humans navigate information but carries little semantic meaning. Moving a document from one folder to another changes its location, but not its intrinsic meaning.

A semantic taxonomy is **fundamentally different.**

Every subclass in an ontology represents a **logical specification** of its parent class. Rather than simply belonging to a parent category, the child concept inherits the semantic meaning established by its ancestors while introducing additional specificity.

For example, the taxonomy introduced in Exercise 14 can be interpreted as follows:



This hierarchy should not be read as:

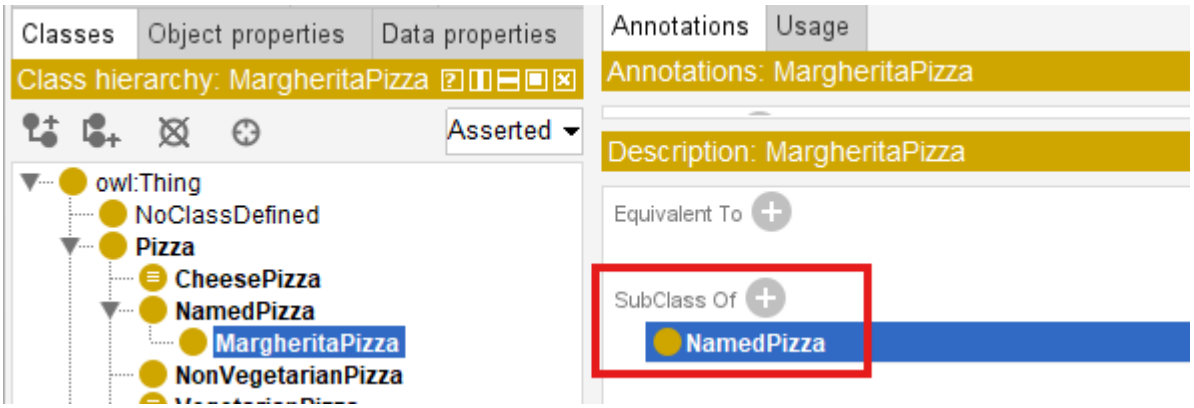
> *`MargheritaPizza` is stored under `NamedPizza`.*

Instead, it should be interpreted as:

> **Every `MargheritaPizza` is a `NamedPizza`, and every `NamedPizza` is a `Pizza`.**

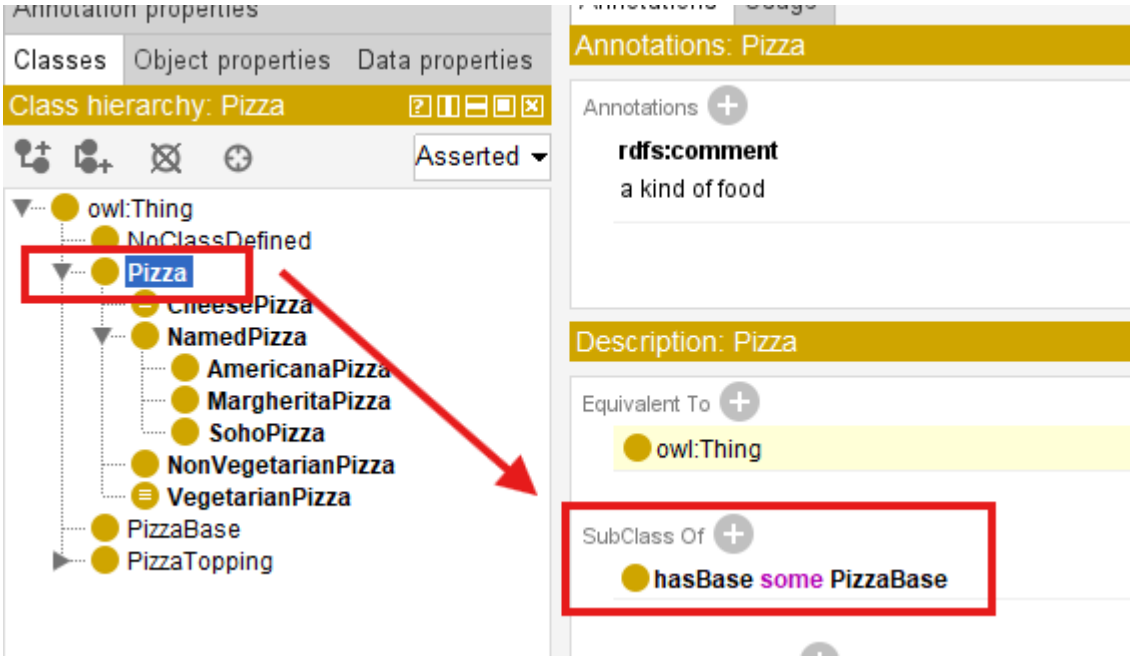
This distinction is subtle but extremely important.

The subclass relationship (`SubClassOf`) expresses an `IS-A` relationship rather than a containment relationship, like below in Protégé:

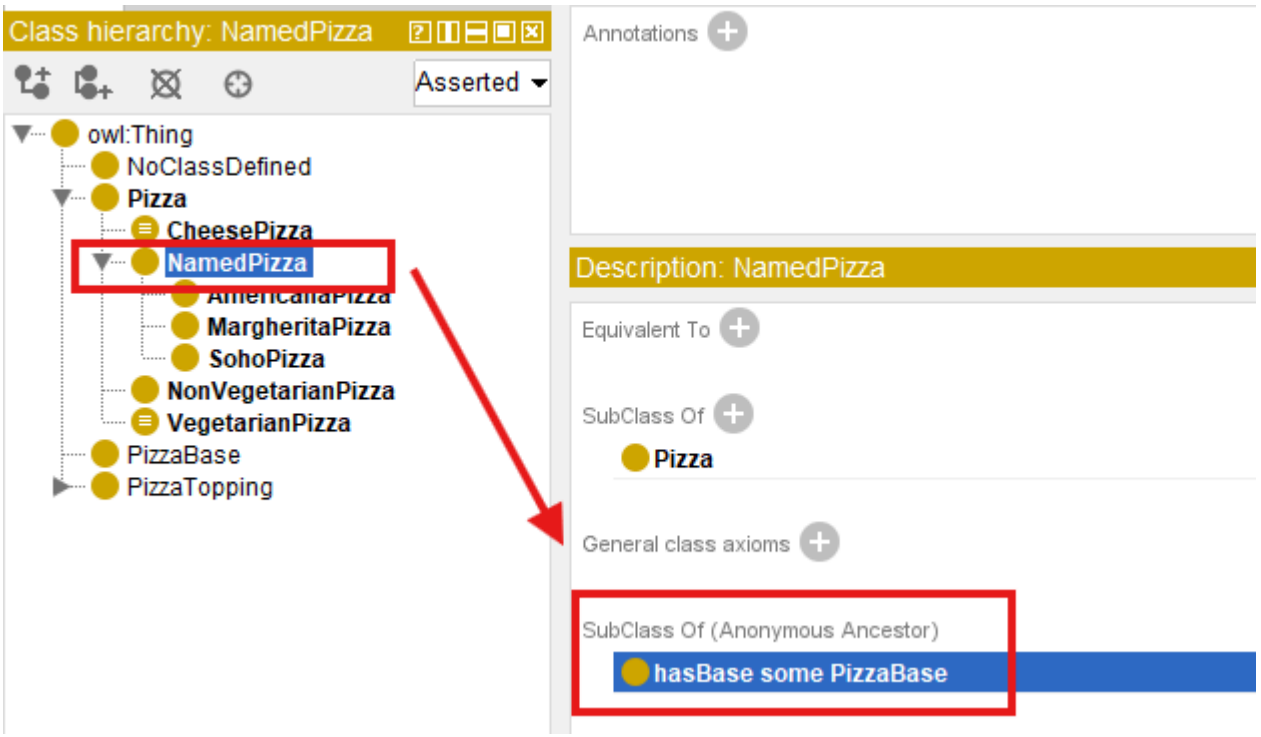


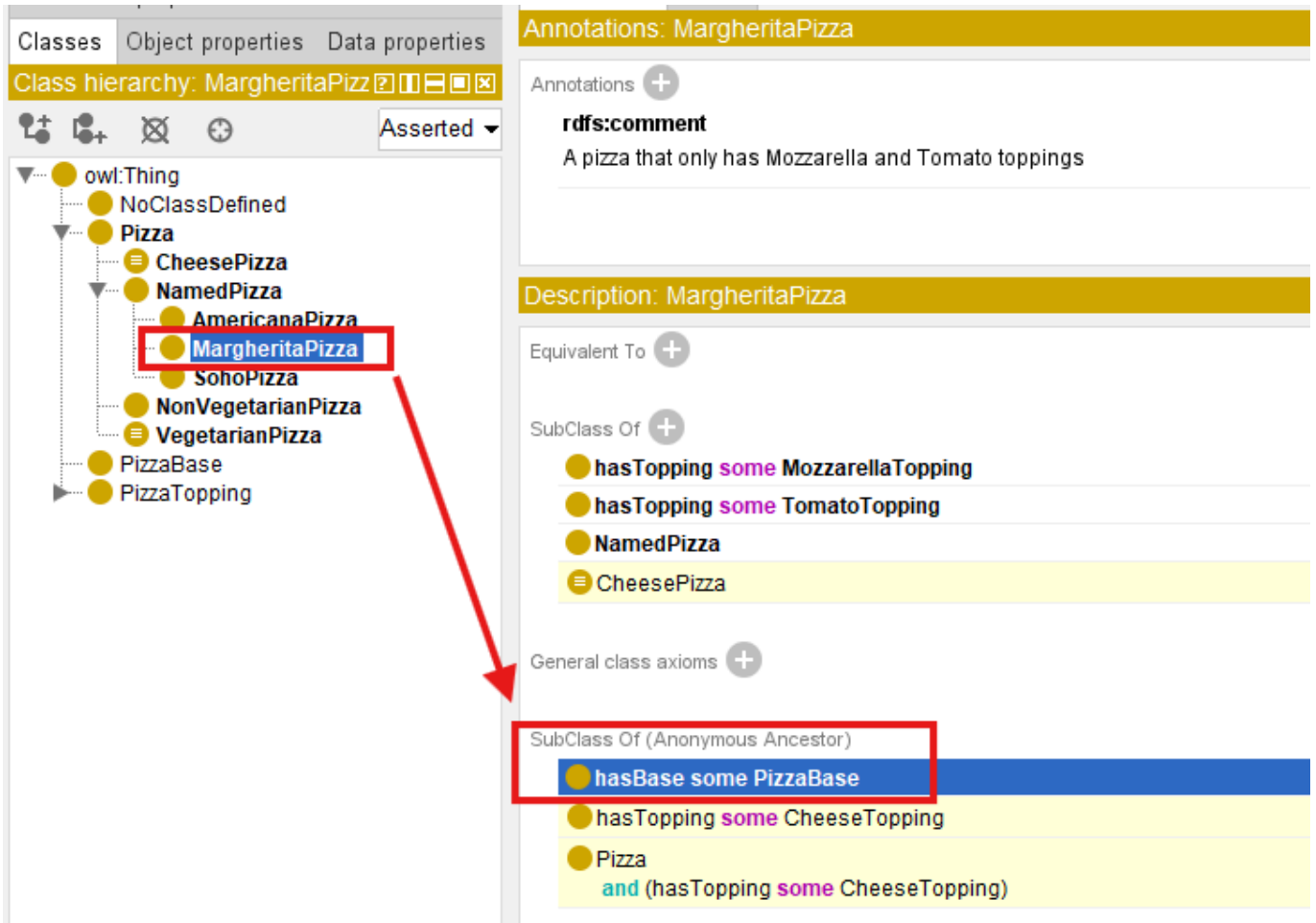
Consequently, semantic properties defined for `Pizza` automatically apply to `NamedPizza` and `MargheritaPizza`, unless explicitly refined or constrained by additional axioms introduced later in the ontology.

Let's see the example, we set: `Pizza hasBase some PizzaBase`:



You don't need separately set this property/constraint, with above taxonomy hierarchy, the `NamedPizza` and `MargheritaPizza` inherit it semantically:





This mechanism is known as **semantic inheritance.**

Unlike object-oriented programming, where subclasses inherit executable methods and implementation details, ontology subclasses inherit **logical meaning**. The purpose is not software reuse, but semantic consistency.

This semantic inheritance enables ontology reasoners to propagate knowledge automatically throughout the class hierarchy, making the taxonomy an active component of semantic reasoning rather than a passive organizational diagram.

For this reason, ontology engineers should never think of the Protégé class hierarchy as a tree of folders. **It is a formal semantic structure that captures conceptual specification within the domain.**

16.7 Interesting Reading -- Mathematical Foundations of Semantic Taxonomy

---

Last updated at: 2026-07-10